# Practical Research Project: Recognition of Road Signs

Martin Drelichowska

University College Roosevelt

DISDATA212 – Image Processing and Computer Vision

prof. dr. ir. Frank van der Stappen

November 30, 2025

**Practical Research Project: Recognition of Road Signs**

## Introduction

The aim of the project is to develop a pipeline of image processing and computer vision algorithms that recognizes prohibition signs A1, C1, C6-C22a, F1, F3, F5, F7, and F10, without determining their specific type. They all have a red annulus in common, so the main goal of the project is to determine whether a red annulus is present in a given image. However, to identify those road signs correctly, it is also necessary to examine the center of the circle. The specified signs can contain only a few colors, that is, white, red, and black, inside the red annulus.

That means there are several steps that need to be taken in order to identify prohibition signs successfully. First, the red elements must be isolated. Then, among the red elements, circles can be detected. For every detected annulus, its inside must be examined to check whether it contains colors that do not occur on the specified road signs.

Because the process is fairly complex, sometimes it is necessary to apply compromise solutions, for example, when deciding what counts as red and what should be classified as background noise and removed. That means there are limitations to the pipeline's accuracy.

The pipeline is based on the Hough Transform used to detect circles. The Hough Transform is a feature extraction method that transforms shapes' representation from the image space to the parameter space, also called the accumulator. The available parameter values for the shapes being detected are represented in the accumulator. A circle is uniquely defined by three parameters: the coordinates of the center $(x, y)$ and the radius $r$, thus the parameter space is 3D.

The Hough Transform iterates through all image coordinates, and if the currently examined point is an edge point, the program iterates through the entire parameter space and checks whether the examined point could fall on the circle defined by the given parameters. If so, the point corresponding to those parameters in the accumulator array gets incremented. At the end, the values from the accumulator that exceed a specific threshold are chosen. The parameters $(x, y$, r) corresponding to those points describe identified circles.

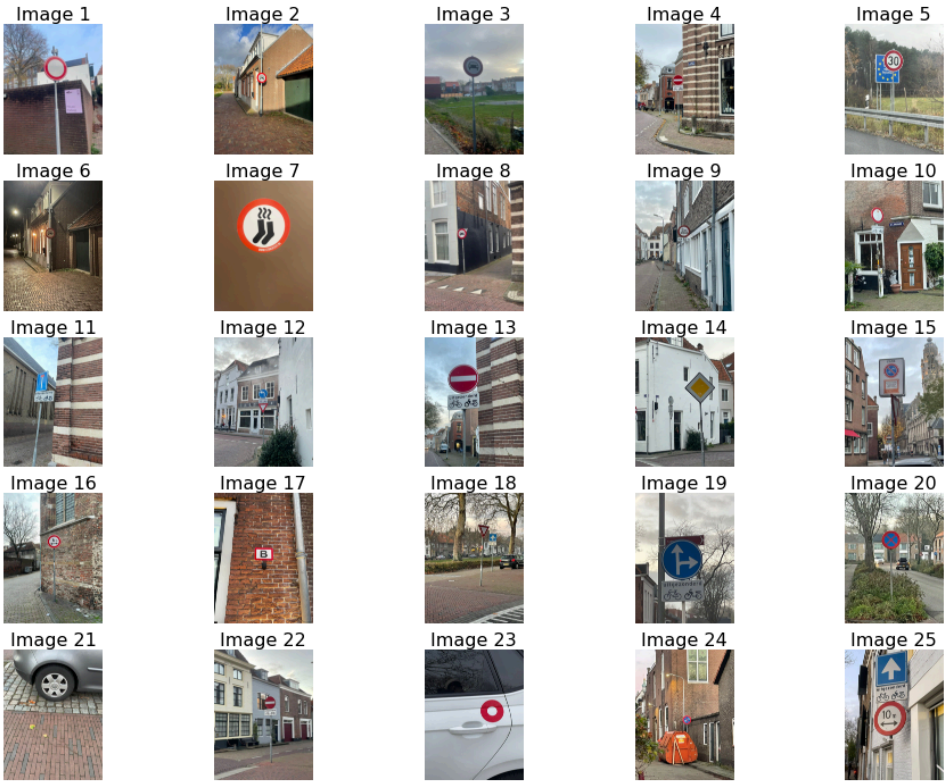
The pipeline has been evaluated on the images taken with a phone camera, as shown in Figure 1.

## Colors

The image is processed in the HSV color space instead of RGB, as for the purpose of this task, HSV is easier to use and more precise than RGB. That is because the hue of a color in RGB is described as a mix of Red, Green, and Blue shades, while in HSV, it is only the parameter H (Hue).

**Figure 1**

*Images used in the pipeline evaluation*




To detect red elements in the RGB color space, a specific value for R above which a pixel could be red has to be defined, but if the values of G or B are too high, the pixel can also be white, yellow, purple, or a shade in between. Therefore, all three channels need to be checked. The boundaries for each are difficult to define, as all the values change significantly depending on the lighting.

That is even more relevant when checking the inside of the circle, as the colors accepted there are red, white, and black. Due to possible differences in brightness between images, white and black can be generalized to shades of gray. Those are difficult to describe using RGB. In the RGB color space, a shade of gray would be described as $(x, x, x)$, but since the images are not perfect, an arbitrary acceptable difference between channels would have to be defined. There is no need to do that in the HSV space, where to check whether a certain pixel is grayscale, it is enough to check its saturation value.

The first step in the pipeline is detecting red elements in a given image. At the beginning, a median blur filter of size $9 \times 9$ is applied to the image to remove the noise. In HSV, the color red "wraps around" the hue circle with values in the range from 0 to 10 and from 160 to 180 representing a red hue. To avoid detecting too bright (almost white), too muted (gray), or too dark (almost black) shades of red, the parameters S and V also need to be defined. For the purpose of this task, red is a color in the range either from (0, 100, 100) to (10, 255, 255) or

in the range from (160, 100, 100) to (179, 255, 255) in the HSV color space. That can be done using the OpenCV function `inRange`. Since there are two boundaries to be evaluated, two results are obtained. Those results can then be merged using the `bitwise_or` function, so that if a pixel is marked as red using either of the two boundaries, it is included in the result, but if it does not fall in the range of either boundary, that pixel does not get counted, and its value in the mask is set to zero. The created mask consists of ones in place of pixels marked as red and zeros in places where red was not detected. Then the image gets added to itself using `bitwise_and` with the created mask serving as a mask, ensuring that colors other than red are set to zero, while red gets highlighted. That step is necessary because some elements of the image are "more red" than others, and taking only the mask as the result would not differentiate between the shades. Figure 2 shows the result of all those operations displayed as a grayscale image. Saturation of the resulting image corresponds to the redness of the elements in the original image. In the HSV color space, isolating the saturation value can easily be done with the OpenCV function `split`. Since there are still elements being detected despite not being purely red but rather orange or brown, as Figure 3 shows, a threshold is applied. If the saturation of a certain pixel in the resulting image is lower than or equal to 140, that pixel is set to zero. All other pixels get set to 255. The result can be observed as removing the noise from the resulting image, as shown in Figure 4. The image after thresholding is a grayscale image of elements that in the original are red. Before circle detection happens, the image gets blurred with a median blur filter of size $5 \times 5$ to mitigate the effects of thresholding on the remaining elements.

**Figure 2**

*Comparison between the original colored image and the result of applying the red mask displayed in grayscale*

**Figure 3**

*Comparison between the original colored image and the result of applying the red mask displayed in grayscale*

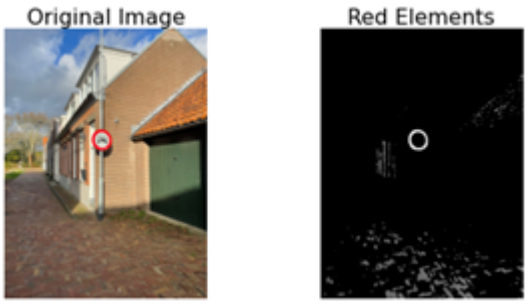
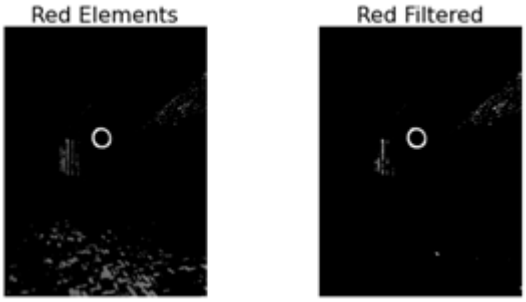


**Figure 4**

*Comparison of the result of applying the red mask to the image before and after thresholding*




**Circle Detection**

The Hough Transform is used to detect circles with the function `HoughCircles`. However, the implementation of the function in OpenCV differs from the standard Hough Transform. It relies on the Hough gradient method, which consists of edge detection and finding possible circle centers, to fit the best radii for each center in the second step.

The parameters in the function are:

```
circles = cv2.HoughCircles(
    s,
    cv2.HOUGH_GRADIENT,
    dp=1,
    minDist=150,
    param1=100,
    param2=30,
    minRadius=10,
    maxRadius=min(s.shape[0], s.shape[1])//2
)
```

where `s` is the final result image from the last step, containing only filtered red elements represented in grayscale. that ensures that only red, not all circles, get detected. Figures 5 and 6 illustrate that and show that the image in which the circles are being detected does not contain non-red elements.

`cv2.HOUGH_GRADIENT` describes the previously mentioned method. The parameter `dp` describes the inverse ratio of the accumulator resolution to the image resolution. It is set to 1, meaning the resolution is the same.

The minimal distance between the centers of circles is 150 pixels to prevent multiple circles from being detected around the true one.

**Figure 5**

*Comparison of the original image, the images with red elements isolated and then thresholded, and the result of circle detection. Non-red circles are not marked.*

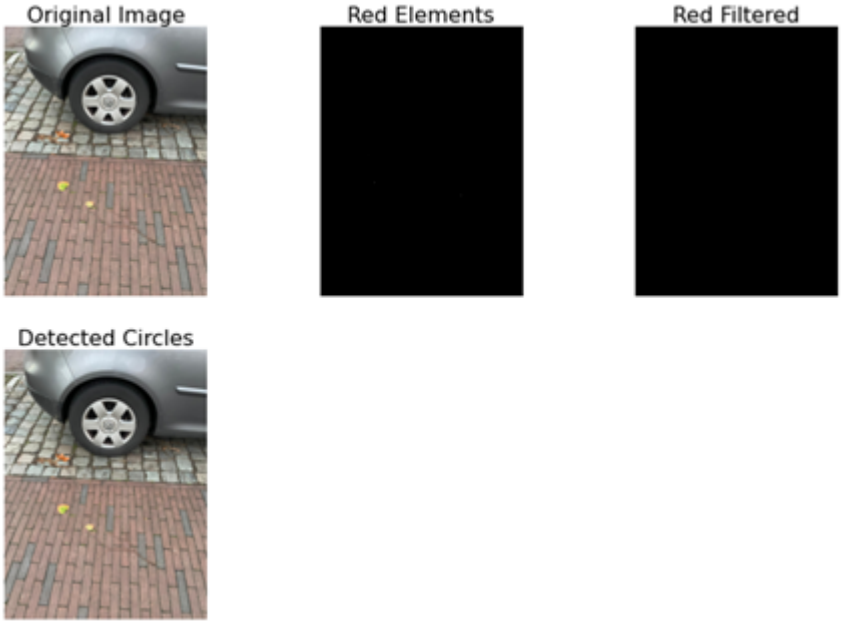
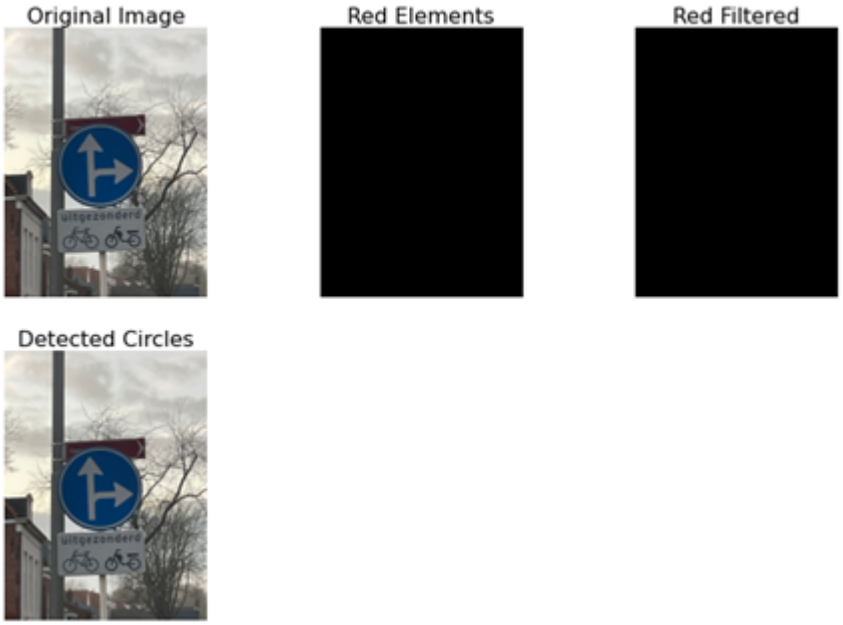



**Figure 6**

*Comparison of the original image, the images with red elements isolated and then thresholded, and the result of circle detection. Non-red circles are not marked*




Based on the `param1` the upper threshold passed to the Canny edge detector is 100, making the lower one 50. Those parameters determine what gets recognized as an edge. Similarly, param2 set to 30 determines what gets recognized as a circle. The values for those parameters are picked by trial and error to minimize the instances of circle detection in noise without compromising too much on accuracy. That way, circles shot at a certain angle can still be recognized, such as the one in Figure 7. Yet, not that much deviance is accepted. Figure 8 shows that if too much of the outline of a circle is missing, it does not get detected at all.

Figure 9 provides an example of a circle shot at an angle too big for it to be accepted. Despite that, there are still circles being detected in the noise or other shapes, as shown by Figures 10 and 11. That is an issue that can be (at least) partially resolved by checking the inside of each circle for any colors that are not supposed to appear inside a prohibition sign.

Additionally, detected circles must have a radius of at least 10 pixels. The maximal radius should not exceed half of the shorter dimension of the considered image, meaning the whole detected circle should be able to fit in the picture, although technically it does not have to. The purpose of those boundaries is to decrease the number of unreasonable matches.

**Figure 7**

*Circle shot at an angle identified as a circle. The result is marked with a green outline.*

**Figure 8**

*The process of color recognition, leading to isolating a circle with missing parts of the outline, prevents the circle from being detected*

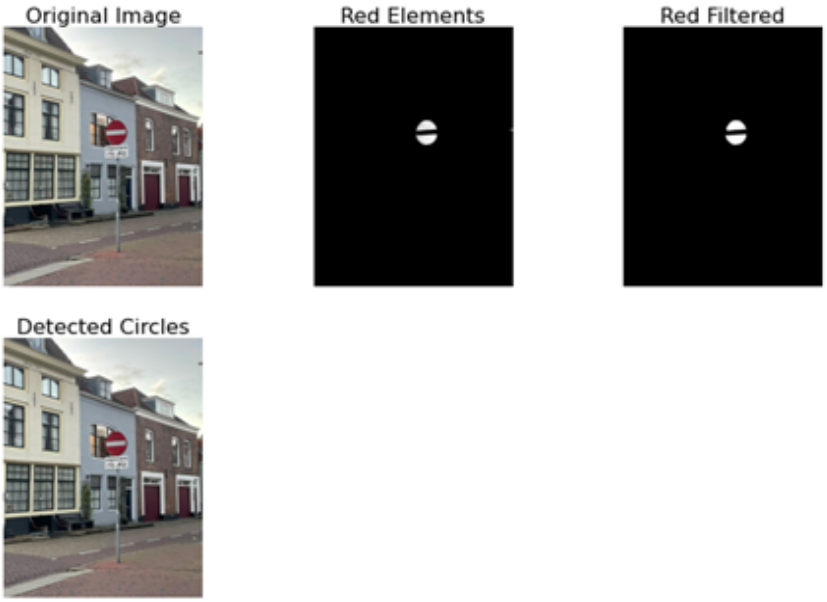


**Figure 9**

*A wide angle from which the photo was taken prevents the circle from being detected*

**Figure 10**

*The result of applying the Hough Transform to a noisy image, where circles (marked with green) are being detected in the noise*

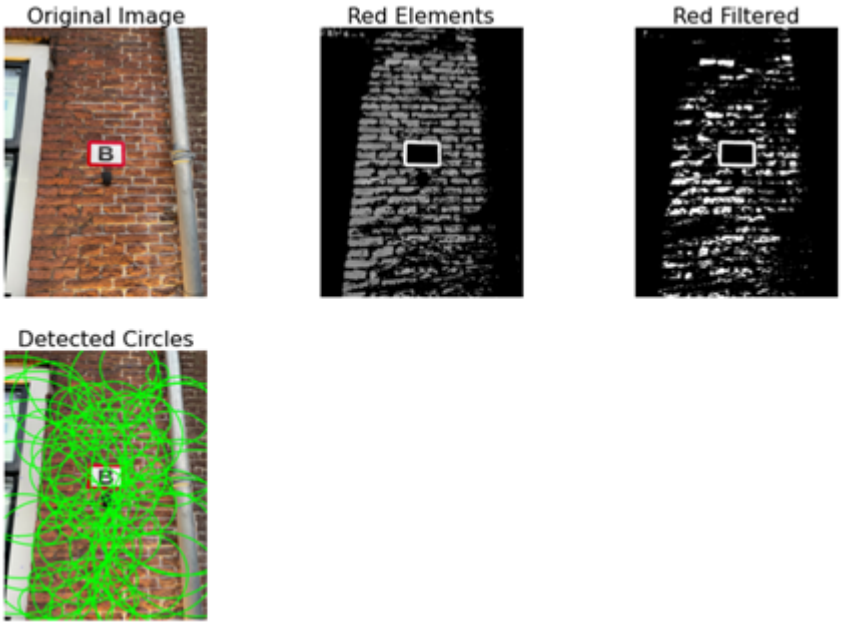
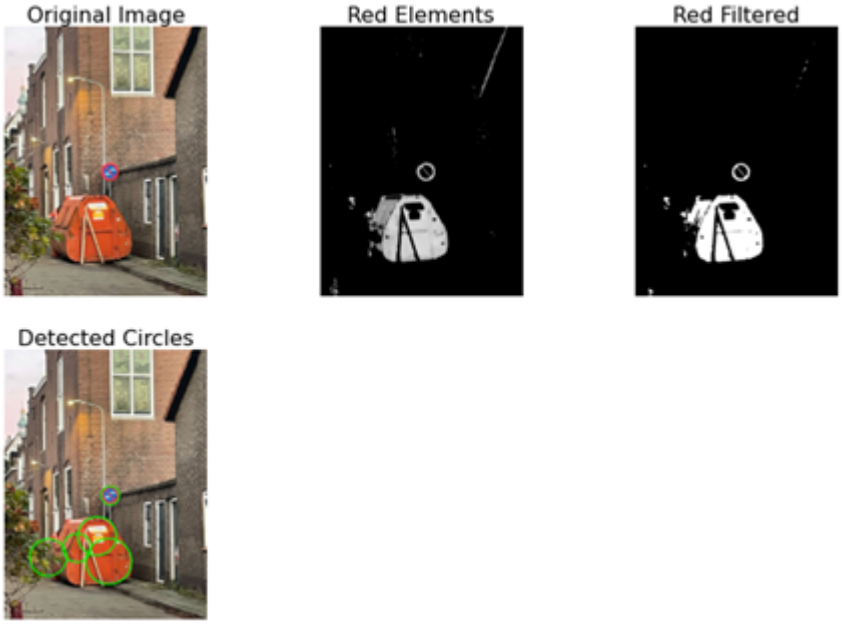


**Figure 11**

*The result of applying the Hough Transform to a noisy image, where circles (marked with green) are being detected in other figures*




The function `HoughCircles` only detects circles, so even if there is a red figure in the image, if it is not a circle, it does not get classified. Figure 12 illustrates that.

**Figure 12**

*The process of red annulus recognition applied to an image containing a red triangle does not result in circle detection*

**Center of the Circle**

The result of `HoughCircles` is an array of arrays of form $(x, y, r)$ describing the coordinates of the center and the radius of each detected circle. That allows for examining the pixels that fall inside each of the detected circles.

If a detected red annulus is fully filled with red or contains colors other than white or black, it cannot possibly be a road sign. There are two problems with these requirements. First of all, while a red circle or the sign C2 contain too much red and should not be detected, C22 is a valid prohibition sign with a red element in the center. Secondly, photos are not perfect, so instead of white and black, shades of gray should be considered.

If a circle contains too much red, it is not a valid prohibition sign and it should be discarded. If it is filled with non-gray colors, it is also not a valid prohibition sign. That can be checked using two loops to examine every pixel that falls inside a circle (and is still in the image). A pixel is considered red if it is marked as such at the end of the color analysis step. It is considered gray if its saturation is lower than 50. Otherwise, it is of a non-gray color. If the ratio of the red area to the total area of the circle or the ratio of the non-gray area to the area of the circle is too large, the circle gets discarded, as it cannot be a valid road sign. The ratio of 0.45 seems to be appropriate for both cases, so it is the value used in the algorithm. Figures 13, 14, and 15 show circles being discarded on the basis of those criteria. Figure 16 shows that if the inside of the circle meets the requirement, the match is kept.

**Figure 13**

*Comparison of all circles detected and circles that meet the prohibition sign requirements. None of the detected circles are prohibition signs*

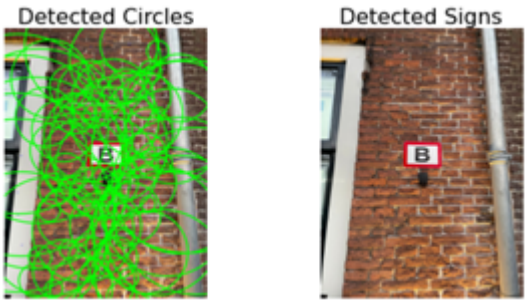



**Figure 14**

*Comparison of all circles detected and circles that meet the prohibition sign requirements. None of the detected circles are prohibition signs*

**Figure 15**

*Comparison of all circles detected and circles that meet the prohibition sign requirements. None of the detected circles are prohibition signs*

**Figure 16**

*Comparison of all circles detected and circles that meet the prohibition sign requirements. Since the detected circle corresponds to a prohibition sign, the match is kept*

**Results and limitations**

Even though the HSV color space is more suited for this task, it is not perfect. Differences in lighting affect colors way too much, making it impossible to define boundaries for the color red that would give satisfactory results in every situation. For example, Figure 17 shows that the red annulus does not get detected in a picture taken at night, because the red in that lighting does not meet the criteria for red. Figure 18 shows a red annulus getting ignored as well because it appears gray under the lighting conditions. There is no way to universally parametrize that without accepting non-red objects as red.

**Figure 17**

*Red elements do not get detected as red under bad lighting conditions*

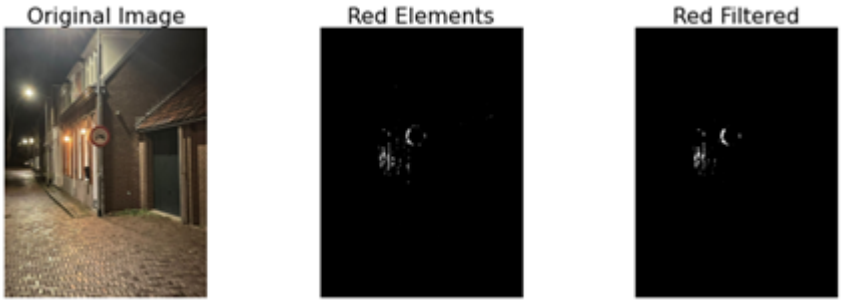
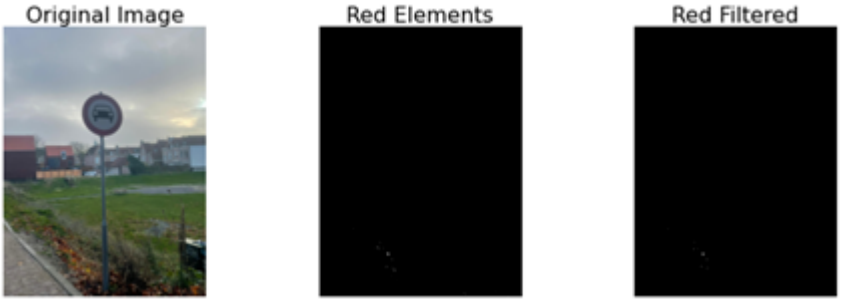


**Figure 18**

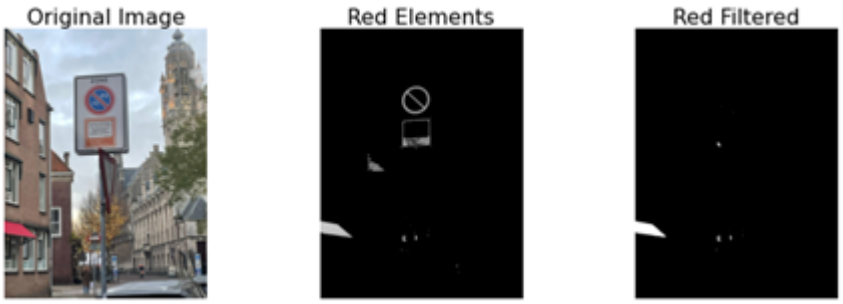
*Red elements do not get detected as red under bad lighting conditions*




Moreover, Figure 19 shows that sometimes elements are correctly marked as red, but because of the lighting conditions, they do not appear red enough to pass the threshold. Again, it is not possible to let every truly red element pass the threshold without letting too much red-ish noise pass as red elements. The thresholding is useful, as shown in Figure 4, because without it, many circles would be detected among the noise. Removing elements that are not red enough reduces the number of false positives, but it also increases the number of false negatives. Yet, reducing the number of false positives is very beneficial for the running time of the algorithm, so it is a decision that seems worth to make. The inside of each detected circle is examined using quite slow algorithm. Reducing the noise leads to fewer false circles being detected and then examined only to be discarded.

**Figure 19**

*Red elements not passing the threshold are removed as noise*




As previously mentioned, there is also the problem with parameters in `HoughCircles`, as some circles shown at an angle or partially covered are not recognized as such. This is also a compromise. Lowering the boundary for classifying a figure as a circle would lead to more false positive hits, and even though they get checked again for the colors on the inside, that is not a perfect test, and the more false positives are being tested, the bigger the chance that some of them will get falsely classified as proper road signs.

The problem of circles shot at an angle not being detected could be solved by looking for ellipses instead of circles. However, an ellipse is uniquely defined by 5 parameters, when a circle can be described using only 3. Consequently, the parameter space for ellipse detection would have to be 5D, which is much less storage efficient.

Most importantly, the algorithm does not actually check whether the image contains a prohibition sign, only whether it is a red annulus filled at most in 45% with red and at most 45% with colors other than shades of gray. The image shown in Figure 20 is not considered a road sign only because the red annulus is too thick compared to the total area of the circle, so there is too much red. The image shown in Figure 21, however, is considered a road sign, despite not being one, as it meets the criteria.

**Figure 20**

*The process of detection of the red annulus and its rejection*

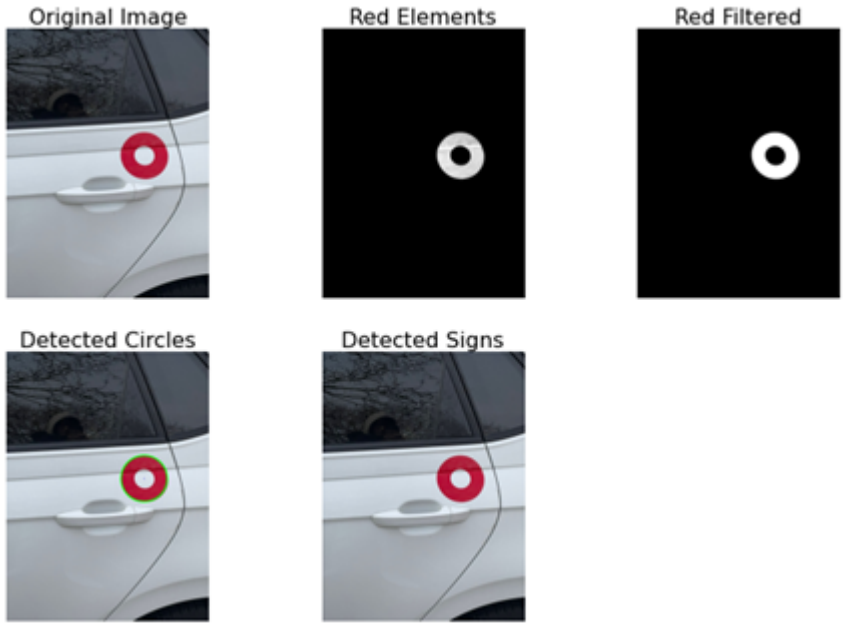
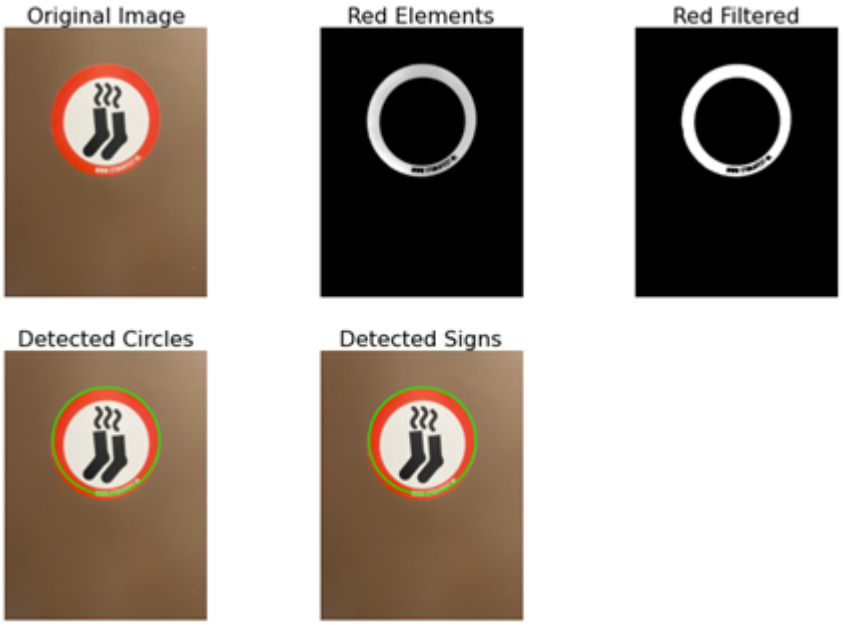



**Figure 21**

*The process of recognition of the red annulus and the result of examining its center*



**Conclusion**

Despite its limitations, a pipeline based on the Hough Transform can be quite efficient in recognizing prohibition signs under certain conditions, such as appropriate lighting and a good angle from which the picture is taken. That is, however, under the assumption that every red annulus that does not contain too many red or non-gray pixels in its center is a prohibition sign, which in practice is not true.

**References**

W. Burger and M. Burge, Principles of Digital Image Processing (Volume 2: Core Algorithms), Springer Undergraduate Topics in Computer Science, New York, 2009
https://answers.opencv.org/question/186189/whats-are-the-values-of-red-in-hsv/
https://www.geeksforgeeks.org/computer-vision/choosing-the-correct-upper-and-lower-hsv-boundaries-for-color-detection-with-cv-inrange-opencv/
https://docs.opencv.org/3.4/dd/d1a/group__imgproc__feature.html#ga47849c3be0d0406ad3ca45db65a25d2d
https://www.geeksforgeeks.org/computer-vision/hough-transform-in-computer-vision/

## Appendix: Python Code

```
import cv2
import matplotlib.pyplot as plt
import numpy as np


def detect_signs(img, x):
    confirmed_signs = []

    nice = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    saved_copy = nice.copy()

    plt.title("Image "+str(x), fontsize = 12, loc='center', pad = 20)
    plt.axis('off')

    plt.subplot(2,3,1)
    plt.imshow(nice)
    plt.axis('off')
    plt.title("Original Image", fontsize = 8, loc='center', pad = 0)

    img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    img = cv2.medianBlur(img, 9)

    lower_bound1 = np.array([0, 100, 100])
    upper_bound1 = np.array([10, 255, 255])
    red1 = cv2.inRange(img, lower_bound1, upper_bound1)

    lower_bound2 = np.array([160, 100, 100])
    upper_bound2 = np.array([179, 255, 255])
    red2 = cv2.inRange(img, lower_bound2, upper_bound2)

    red = cv2.bitwise_or(red1, red2)

    final = cv2.bitwise_and(img, img, mask=red)

    h, s, v = cv2.split(final)
    plt.subplot(2,3,2)
    plt.imshow(s, cmap = 'gray')
    plt.axis('off')
    plt.title("Red Elements", fontsize = 8, loc='center', pad = 0)

    _, s =  cv2.threshold(s, 140, 255, cv2.THRESH_BINARY)

    plt.subplot(2,3,3)
    plt.imshow(s, cmap = 'gray')
    plt.axis('off')
    plt.title("Red Filtered", fontsize = 8, loc='center', pad = 0)

    s = cv2.medianBlur(s, 5)

    circles = cv2.HoughCircles(
        s,
        cv2.HOUGH_GRADIENT,
        dp=1,
        minDist=150,
        param1=100,
        param2=30,
```

```
        minRadius=10,
        maxRadius=min(s.shape[0], s.shape[1])//2
    )

    if circles is not None:
        circles = np.int32(np.around(circles))
        for i in circles[0, :]:
            cv2.circle(nice, (i[0], i[1]), i[2], (0, 255, 0), 10)
            cv2.circle(nice, (i[0], i[1]), 2, (0, 0, 255), 3)

            red = s.copy()
            h1, s1, v1 = cv2.split(img)
            x = i[0]
            y = i[1]
            r = i[2]

            red_pixels = 0
            other_colors = 0
            total = 0

            for a in range(x-r, x+r):
                for b in range(y - r, y + r):
                    if min(x-r, y-r) >= 0 and x+r < img.shape[1] and y+r <
img.shape[0]:
                        if (a-x)**2 + (b-y)**2 <= r**2:
                            total += 1
                            if red[b][a] == 255:
                                red_pixels += 1
                            elif s1[b][a] > 50:
                                other_colors += 1
            """
            if red_pixels > 0.45 * total:
                print("too much red")

            elif other_colors > 0.45 * total:
                print("weird colors in the circle")
            else:
                #print(other_colors/total)
                confirmed_signs.append((x, y, r))
            """
            if red_pixels < 0.45 * total and other_colors < 0.45 * total:
                confirmed_signs.append((x, y, r))

    plt.subplot(2,3,4)
    plt.imshow(nice)
    plt.axis('off')
    plt.title("Detected Circles", fontsize = 8, loc='center', pad = 0)

    canvas = saved_copy.copy()
    for each in confirmed_signs:
        x = each[0]
        y = each[1]
        r = each[2]
        cv2.circle(canvas, (x, y), r, (0, 255, 0), 10)

    plt.subplot(2,3,5)
    plt.imshow(canvas)
    plt.axis('off')
    plt.title("Detected Signs", fontsize = 8, loc='center', pad = 0)
```

```
    plt.show()

fig = plt.figure(figsize=(8, 6))

for x in range(1, 26):
    path = "img" + str(x)+".jpg"
    img = cv2.imread(path, cv2.IMREAD_COLOR)
    nice = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    plt.subplot(5, 5, x)
    plt.imshow(nice)
    plt.axis('off')
    plt.title("Image "+str(x), fontsize = 8, loc='center', pad = 0)
plt.show()


for x in range(1, 26):
    path = "img" + str(x)+".jpg"
    img = cv2.imread(path, cv2.IMREAD_COLOR)
    detect_signs(img, x)
```